LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 4
Student Name: K. Sirisha
Roll No.: A24126552260
Section: B

---

1. TITLE

Interactive To-Do List Using DOM and Event Listeners

2. AIM

To develop an interactive To-Do List web application that lets a user add, complete and delete tasks using DOM manipulation and event listeners.

3. OBJECTIVE

The objectives of this experiment are:

- To dynamically create and insert list elements into the DOM.
- To capture user input from a text field and a button click.
- To attach event listeners for click and keyboard events.
- To toggle a task's completed state and update its style accordingly.
- To remove a task element from the DOM when deleted.
- To keep a live summary of total, completed and pending tasks.

4. THEORY

Event listeners let JavaScript respond to user actions on the page. The method `element.addEventListener(event, handler)` attaches a function that runs whenever the specified event — such as `"click"` or `"keydown"` — occurs on that element.

DOM manipulation methods such as `document.createElement()`, `appendChild()` and `removeChild()` allow elements to be created and removed from the page at runtime, without a reload. The `classList.toggle()` method adds or removes a CSS class from an element, which is used here to switch a task between its normal and "completed" (strikethrough) appearance.

In this application, each task is represented by an `<li>` element containing a checkbox, a text label and a delete button, all created dynamically and connected to their own event listeners when the task is added.

5. ALGORITHM / PROCEDURE

1. Create a new HTML file with a text input, an "Add Task" button and an empty unordered list.
2. Write a `createTask(text, completed)` function that builds an `<li>` containing a checkbox, a text span and a delete button.
3. Inside `createTask()`, attach a `click` event listener to the checkbox and text span that toggles the `completed` class.
4. Attach a `click` event listener to the delete button that removes the `<li>` using `removeChild()`.
5. Append the completed `<li>` to the task list using `appendChild()`.
6. Write an `addTaskFromInput()` function that reads the input value and calls `createTask()`, then clears the input.
7. Attach a `click` event listener to the "Add Task" button that calls `addTaskFromInput()`.
8. Attach a `keydown` event listener to the input field that also calls `addTaskFromInput()` when the Enter key is pressed.
9. Write an `updateSummary()` function that counts total and completed tasks and updates a summary line.
10. Pre-load a few sample tasks on page load to demonstrate the rendering.
11. Save the file, open it in a browser, and add, complete and delete tasks to verify the behaviour.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Interactive To-Do List</title>
<style>
  body { font-family: Arial, sans-serif; background: #f4f6fa; margin: 0; }
  header { background: #1a3c6e; color: #fff; padding: 18px 25px; }
  header h1 { margin: 0; font-size: 24px; }
  main { max-width: 520px; margin: 25px auto; padding: 0 20px; }

  .input-row { display: flex; gap: 10px; }
  #taskInput { flex: 1; padding: 10px; border: 1px solid #bbb; border-radius: 5px; font-size: 14px; }
  #addBtn { background: #1a3c6e; color: #fff; border: none; padding: 10px 18px; border-radius: 5px; cursor: pointer; font-size:
14px; }
  #addBtn:hover { background: #2e5aa8; }

  ul#taskList { list-style: none; padding: 0; margin-top: 18px; }
  ul#taskList li {
    background: #fff; border: 1px solid #ddd; border-radius: 6px;
    padding: 10px 14px; margin-bottom: 8px; display: flex; align-items: center;
    justify-content: space-between; box-shadow: 0 1px 4px rgba(0,0,0,0.07);
  }
  ul#taskList li span.text { flex: 1; margin-left: 8px; cursor: pointer; }
  ul#taskList li.completed span.text { text-decoration: line-through; color: #888; }
  li button.remove { background: #c0392b; color: #fff; border: none; border-radius: 4px; padding: 4px 10px; cursor: pointer; font-
size: 12px; }
  li button.remove:hover { background: #e74c3c; }

  #summary { margin-top: 14px; font-size: 13px; color: #555; }
</style>
</head>
<body>

<header>
  <h1>Interactive To-Do List</h1>
</header>

<main>
  <div class="input-row">
    <input type="text" id="taskInput" placeholder="Enter a new task...">
    <button id="addBtn">Add Task</button>
  </div>

  <ul id="taskList"></ul>
  <p id="summary"></p>
</main>

<script>
  const taskInput = document.getElementById("taskInput");
  const addBtn = document.getElementById("addBtn");
  const taskList = document.getElementById("taskList");
  const summary = document.getElementById("summary");

  function updateSummary() {
    const total = taskList.children.length;
    const done = taskList.querySelectorAll("li.completed").length;
    summary.textContent = `Total tasks: ${total}  |  Completed: ${done}  |  Pending: ${total - done}`;
  }

  function createTask(text, completed) {
    const li = document.createElement("li");
    if (completed) li.classList.add("completed");

    const checkbox = document.createElement("input");
    checkbox.type = "checkbox";
    checkbox.checked = completed;

    const span = document.createElement("span");
    span.className = "text";
    span.textContent = text;

    const removeBtn = document.createElement("button");
    removeBtn.className = "remove";
    removeBtn.textContent = "Delete";

    // Event listener: toggle complete on checkbox click
    checkbox.addEventListener("click", function () {
      li.classList.toggle("completed");
      updateSummary();
    });

    // Event listener: toggle complete on text click too
    span.addEventListener("click", function () {
      checkbox.checked = !checkbox.checked;
      li.classList.toggle("completed");
```

```
      updateSummary();
    });

    // Event listener: delete task
    removeBtn.addEventListener("click", function () {
      taskList.removeChild(li);
      updateSummary();
    });

    li.appendChild(checkbox);
    li.appendChild(span);
    li.appendChild(removeBtn);
    taskList.appendChild(li);
    updateSummary();
  }

  function addTaskFromInput() {
    const value = taskInput.value.trim();
    if (value === "") return;
    createTask(value, false);
    taskInput.value = "";
    taskInput.focus();
  }

  // Event listener: Add button click
  addBtn.addEventListener("click", addTaskFromInput);

  // Event listener: Enter key inside input
  taskInput.addEventListener("keydown", function (e) {
    if (e.key === "Enter") {
      addTaskFromInput();
    }
  });

  // Pre-load sample tasks to demonstrate functionality
  createTask("Complete Full Stack lab record", false);
  createTask("Submit assignment before deadline", false);
  createTask("Revise JavaScript DOM concepts", true);
  createTask("Attend lab viva session", true);
</script>

</body>
</html>
```

7. CODE EXPLANATION

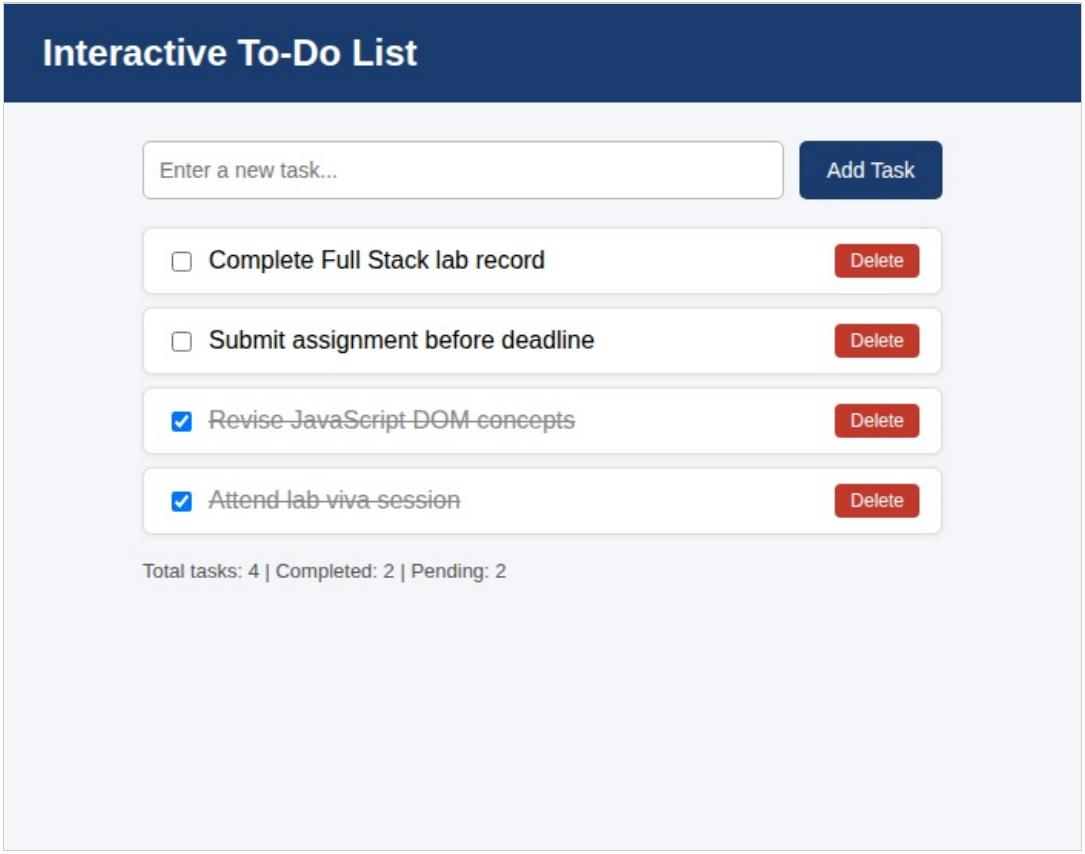
Code	Explanation
`document.createElement("li")`	Creates a new list item element to represent one task.
`checkbox.addEventListener("click", ...)`	Toggles the task's completed style when the checkbox is clicked.
`span.addEventListener("click", ...)`	Also toggles completion when the task text itself is clicked.
`removeBtn.addEventListener("click", ...)`	Removes the task's `<li>` from the list using removeChild().
`classList.toggle("completed")`	Adds/removes the CSS class that applies the strikethrough style.
`addBtn.addEventListener("click", ...)`	Adds a new task when the "Add Task" button is clicked.
`taskInput.addEventListener("keydown", ...)`	Adds a new task when the Enter key is pressed inside the input field.
`updateSummary()`	Recalculates and displays the total, completed and pending task counts.

8. TEST CASES AND EXECUTION DATA

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Load the page in browser	4 sample tasks are shown, 2 marked completed	Displayed correctly, summary read 4 total / 2 completed	**Pass**
TC-02	Type "Prepare for upcoming viva" and press Enter	New task is added to the bottom of the list and input clears	Task added, input cleared, total became 5	**Pass**
TC-03	Click the checkbox of a pending task	Task text gets a strikethrough style and summary updates	Strikethrough applied, completed count increased	**Pass**
TC-04	Click "Delete" on a task	That task is removed from the list and summary updates	Task removed, totals updated correctly	**Pass**
TC-05	Click "Add Task" with the input left empty	No empty task should be added	No task was added	**Pass**

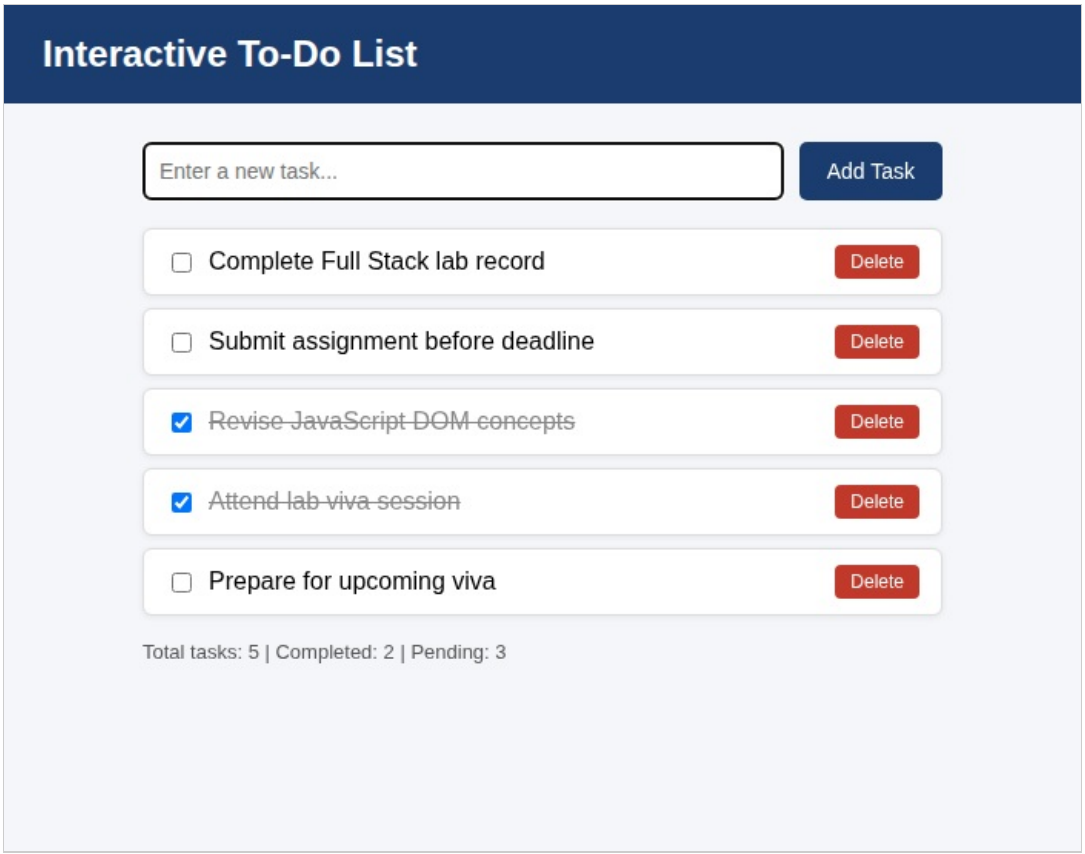
9. OUTPUT

Output 1: Initial To-Do List with Pre-loaded Tasks



Rendered in browser — 4 sample tasks, with 2 marked completed (strikethrough) and a live summary of total/completed/pending tasks.

Output 2: After Adding a New Task via the Enter Key



A new task "Prepare for upcoming viva" was typed and added using the keydown event listener; the summary line updated to 5 total tasks.

10. RESULT

Thus, an Interactive To-Do List web application was successfully developed using DOM manipulation and event listeners, allowing tasks to be added, marked complete and deleted dynamically. The output was verified against the expected results for all test cases.